

Hanoi University of Science and Technology
School of Information and Communications Technology



SOICT

Applied Statistics and Experimental Design

PROJECT REPORT

Time series noise reduction

Nguyen Duc Dung

Data Science & AI

HUST

`dung.nd2416681@hust.sis.edu.vn`

Nguyen Hoang Quan

Data Science & AI

HUST

`quan.nh2416738@sis.hust.edu.vn`

Nguyen Cong Hung

Data Science & AI

HUST

`hung.nc2416702@sis.hust.edu.vn`

Nguyen Tuan Son

Data Science & AI

HUST

`son.nt2416748@sis.hust.edu.vn`

Le Trong Dat

Data Science & AI

HUST

`dat.lt2416672@sis.hust.edu.vn`

Course: Applied Statistics and Experimental Design

Class ID: 168274

Semester: 20252

Supervisor: Dr. Doan Phong Tung

Hanoi, 2026

Table of Contents

1	Introduction	4
2	Related work	5
3	Data Analytics	6
3.1	Overview	6
3.1.1	MIT-BIH Database	6
3.1.2	FPT Stock Database	7
3.2	Explorational Data Analysis	7
3.2.1	MIT BIH Dataset	7
3.2.2	FPT Stocks Datasets	9
4	Background Preliminaries	14
4.1	Filter-based Methods	14
4.1.1	Median Filter	14
4.1.2	Kalman Filter	15
4.1.3	Spectral Gating	16
4.2	Seasonal and Trend Decomposition Using Loess (STL)	17
4.3	Auto Regressive Integrated Moving Average (ARIMA)	19
4.4	Long Short-Term Memory (LSTM)	20
5	Experiment Setup	21
5.1	MIT-BIH Database	21
5.1.1	Backbone Model Architecture	21
5.1.2	Overall Pipeline	22
5.1.3	Denoising and Trend Extraction Modules	23
5.2	FPT Stocks Database	27
5.2.1	Dataset and Labeling Strategy	27
5.2.2	Backbone Model Architecture	27
5.2.3	Overall Pipeline	28
5.2.4	Denoising and Trend Extraction Modules	28
5.2.5	Training Hyperparameters Details	32
5.3	Metrics Benchmarks	32
6	Experiment Result	33
7	Conclusion	35

Abstract

Time series data collected in practical settings are often contaminated by noise arising from sensor limitations, environmental disturbances, or irregular measurement processes. Such noise can obscure underlying structures, weaken interpretability, and reduce the reliability of subsequent forecasting or decision-making tasks. This report investigates several representative approaches to time series noise reduction, with emphasis on both classical filtering techniques and statistical modeling methods.

We first review filter-based methods, including the median filter, Kalman filter, and spectral gating, highlighting their intuition, strengths, and limitations in different noise conditions. We then examine decomposition-based and model-based approaches, namely Seasonal and Trend decomposition using Loess (STL) and ARIMA, to illustrate how trend, seasonality, and stochastic components can be separated or modeled for denoising purposes. By comparing these methods conceptually and analytically, the report aims to provide a clear perspective on when each technique is most appropriate.

Through this study, we show that effective noise reduction is not only a preprocessing step but also a crucial part of extracting meaningful information from temporal data. The discussion therefore serves as a concise foundation for researchers and students interested in robust time series analysis.

Code available at: https://github.com/DucDung-1107/IT2022_Projects

1 Introduction

Time series are a fundamental data representation in many scientific and engineering domains, where observations are collected sequentially over time and the order of measurements carries important information. In practice, however, time series are rarely observed in a clean form. Biomedical recordings may be affected by sensor artifacts, baseline drift, electrode motion, and environmental interference, while financial time series are influenced by market microstructure noise, abrupt liquidity changes, and irregular volatility regimes. These disturbances can obscure the underlying dynamics of the system, distort statistical summaries, and reduce the reliability of downstream tasks such as forecasting, anomaly detection, classification, and decision making.

This report studies the problem of **time series noise reduction**. The central objective is to transform an observed noisy sequence into a cleaner representation that preserves meaningful temporal structure while suppressing irrelevant fluctuations. This task is challenging because noise is not universal: it may appear as isolated spikes, high-frequency oscillations, slow baseline drift, stochastic measurement error, or regime-dependent volatility. As a result, no single denoising method is optimal for all settings. A robust analysis must therefore consider both the statistical properties of the data and the assumptions made by each filtering or modelling technique.

To ground the discussion in realistic applications, the report considers two datasets with substantially different characteristics. The first is the MIT-BIH Arrhythmia Database, a benchmark electrocardiogram (ECG) dataset used in biomedical signal processing. ECG signals contain sharp morphological components such as QRS complexes, together with possible baseline wander and local measurement noise. The second is a historical FPT stock dataset from the Vietnamese equity market, where the closing price, return, and trading volume series exhibit non-stationarity, volatility clustering, and liquidity shocks. These two datasets provide complementary case studies: one represents high-frequency physiological signals with interpretable waveform structure, while the other represents financial time series with stochastic trends and market-driven uncertainty.

The report investigates several representative approaches to noise reduction. Filter-based methods, including the median filter, Kalman filter, and spectral gating, are reviewed as direct preprocessing tools for removing impulsive noise, recursive measurement noise, and frequency-domain disturbances. Decomposition-based and statistical modelling approaches, including STL and ARIMA, are then examined to show how trend, seasonality, autocorrelation, and residual noise can be separated. Finally, Long Short-Term Memory (LSTM) networks are discussed as a data-driven approach capable of learning nonlinear and long-range temporal dependencies. By comparing these methods conceptually and analytically, the report aims to clarify their assumptions, strengths, limitations, and suitability for different types of time series.

2 Related work

Classical time series analysis provides the theoretical foundation for many noise reduction methods. Box–Jenkins modelling formalized the use of autoregressive and moving-average structures for representing serial dependence, with differencing used to handle non-stationary signals [2]. ARIMA models remain widely used because they offer interpretable parameters, residual diagnostics, and principled order selection through information criteria such as AIC and BIC [1, 11]. In denoising applications, ARIMA can be interpreted as a model-based filter: predictable autocorrelated components are retained, while residual innovations are treated as noise. However, ARIMA assumes a largely linear structure and usually requires stationarity after differencing, which limits its flexibility for nonlinear or strongly regime-dependent data.

Filtering methods form another major branch of time series denoising. The median filter is a simple nonlinear technique that replaces each observation with the median of a local window, making it effective against impulsive spikes while preserving sharp edges better than moving averages. Kalman filtering provides a recursive probabilistic framework for estimating hidden states from noisy observations under a state-space model [7]. Its strength lies in online updating and uncertainty-aware estimation, but its standard form relies on linear-Gaussian assumptions; nonlinear extensions and practical variants have therefore been developed for more complex systems [8, 9]. Spectral methods, including spectral gating, instead operate in the frequency domain and suppress components whose energy is likely to correspond to noise. These methods are especially useful when signal and noise occupy different frequency bands, but they may distort signals whose informative content overlaps with the noise spectrum.

Decomposition-based approaches seek to separate a time series into interpretable components such as trend, seasonality, and remainder. STL is a prominent example that uses LOESS smoothing to extract seasonal and trend components while leaving an irregular residual component. Its robustness and interpretability make it useful for exploratory analysis and pre-processing, particularly when the data contain repeated seasonal structure. Nevertheless, STL depends on appropriate smoothing and seasonal-period choices, and it is less suitable for signals without stable periodicity. For financial data such as stock prices, decomposition is often complicated by stochastic trends, changing volatility, and structural breaks, so stationarity tests and residual diagnostics remain essential [5, 13].

Recent work has increasingly adopted machine learning methods for time series denoising, especially recurrent neural networks. LSTM networks are designed to address the vanishing-gradient problem in standard recurrent models by using gated memory cells, allowing them to capture long-range dependencies and nonlinear temporal patterns. In the context of noise reduction, an LSTM can learn to reconstruct a clean signal from noisy observations or to predict the structured component of a sequence while leaving unpredictable residuals as noise. Compared with classical models, LSTM methods are more flexible and can incorporate multivariate inputs, but they require larger datasets, careful regularization, and more computational resources. They also provide less direct interpretability than ARIMA, STL, or Kalman filtering. Therefore, this report treats LSTM not as a replacement for classical methods, but as a

complementary approach whose usefulness depends strongly on data availability and modelling objectives.

3 Data Analytics

3.1 Overview

3.1.1 MIT-BIH Database

The MIT-BIH Arrhythmia Database is one of the most important and widely used ECG datasets in the world for arrhythmia research. Created jointly by MIT and Beth Israel Hospital (now Beth Israel Deaconess Medical Center), the database was completed in 1980 and made fully available on PhysioNet in 2005. It contains 48 half-hour excerpts of two-channel ambulatory ECG recordings from 47 subjects, collected between 1975 and 1979. Of these, 23 records were randomly selected from a collection of 4000 ambulatory ECGs, while the remaining 25 were deliberately chosen to include less common but clinically significant arrhythmias. The signals were digitized at 360 Hz with 11-bit resolution over a 10 mV range, and include approximately 110,000 beat annotations carefully verified by cardiologists. This dataset has served as the standard benchmark for evaluating arrhythmia detection algorithms for over four decades and remains a cornerstone resource in biomedical signal processing and machine learning research.

Properties:

- Contains 48 records (numbered from 100 to 124, excluding some numbers), each representing a 30-minute two-channel ECG recording.
- Total dataset includes approximately 110,000 beat annotations across all records.
- Each record consists of two ECG channels: MLII (Modified Limb Lead II) and V5 (Chest Lead V5).
- Signals are sampled at 360 Hz with 11-bit resolution over a 10 mV range.
- Raw signal files are provided in .csv format containing columns: sample #, MLII, and V5.
- Annotation files include:
 - `annotations.txt`: Human-readable format with columns (Time, Sample #, Type, Sub, Chan, Num, Aux).
 - `.atr` files: Binary annotation files.
- Beat types are annotated using standard symbols (e.g., N = Normal beat, A = Atrial premature, V = Ventricular premature, etc.).

- Each record has a corresponding annotation file providing precise timing and classification for every heartbeat.

3.1.2 FPT Stock Database

The FPT Stock Dataset comprises historical daily trading data of FPT Corporation (ticker: FPT) listed on the Ho Chi Minh Stock Exchange (HOSE). The dataset spans from August 24, 2017 to April 9, 2025, covering more than 7.5 years of trading activity. It consists of five main features: Date (trading day), Open (opening price), High (highest price), Low (lowest price), and Close (closing price), all denominated in Vietnamese Dong (VND). This dataset serves as a valuable resource for time series forecasting, stock price prediction, volatility analysis, and financial modeling, particularly in studies related to Vietnam’s technology sector and stock market.

Properties

- Contains historical daily trading records of FPT Corporation listed on HOSE.
- Covers the period from August 24, 2017 to April 9, 2025, representing more than 7.5 years of market activity.
- Each row corresponds to one trading day, making the dataset suitable for daily time series analysis and forecasting.
- Includes six core attributes: Date, Open, High, Low, Close, and Volume.
- All price variables are measured in Vietnamese Dong (VND).
- The Open feature records the opening price, while Close gives the final transaction price of the trading session.
- The High and Low variables capture the maximum and minimum prices reached within each trading day.
- The Volume feature records the total number of shares traded during each session, providing a useful indicator of market liquidity and trading intensity.

3.2 Explorational Data Analysis

3.2.1 MIT BIH Dataset

Signal quality and temporal structure. Record 100 (MLII, first 20 seconds) shows three main patterns: a mostly stable ECG baseline with mild low-frequency drift, periodic variance spikes aligned with QRS complexes, and a differenced signal that is approximately zero-mean. These observations indicate that the segment is relatively clean, rhythmically regular, and mainly affected by weak baseline drift rather than severe artifacts.

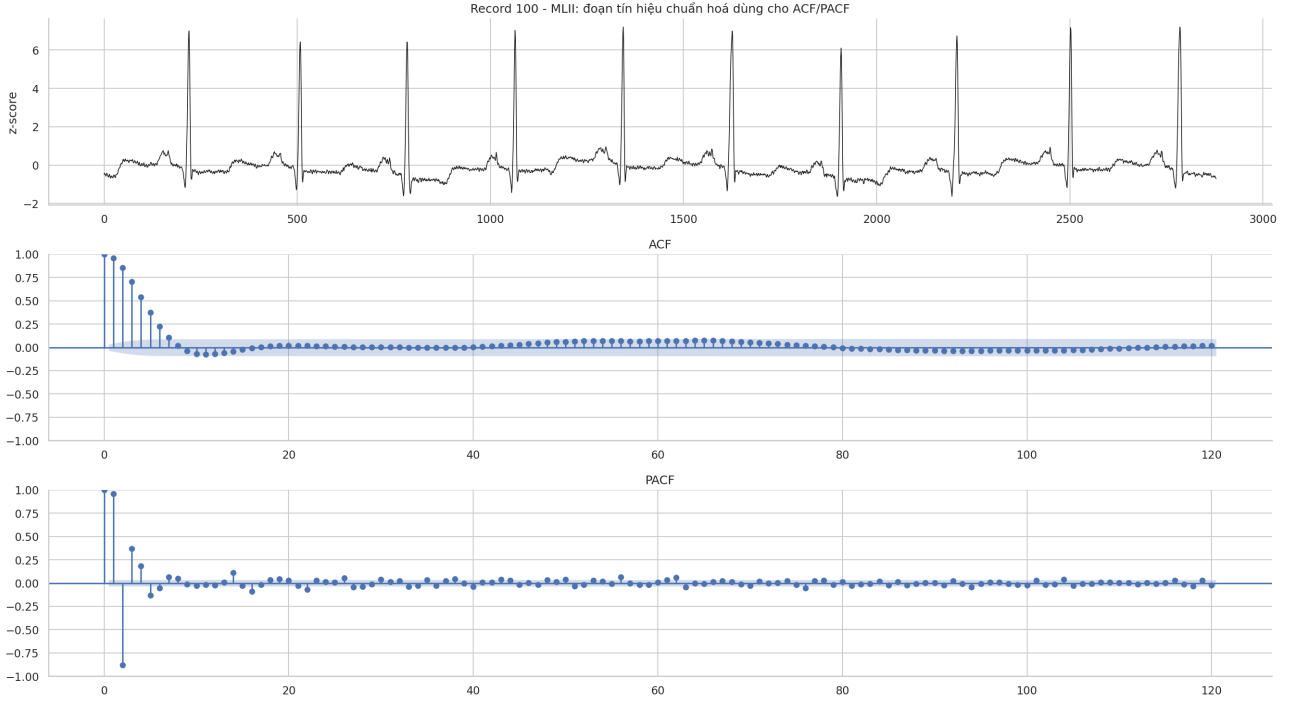


Figure 1: Record 100 – MLII: raw signal with rolling mean, rolling standard deviation, and first-order difference.

After z-score normalisation, R-peaks become clear high-amplitude anchors for beat segmentation. The ACF and PACF plots further show that most dependence is short-range, with the strongest structure concentrated in the first one or two lags. This supports the use of simple autoregressive modelling as a local approximation for short ECG segments.

Stationarity tests. The ADF–KPSS results suggest that the raw ECG is not dominated by a stochastic unit root, but it still contains weak deterministic drift. After first-order differencing, both tests support stationarity. This confirms that differencing is sufficient to stabilize the short ECG segment for local time-series modelling.

Overall, the MIT-BIH analysis suggests three practical preprocessing insights. First, the ECG signal is only weakly trend-stationary, so classical time-series models benefit from detrending, high-pass filtering, or first-order differencing. Second, the PACF pattern shows that short-range dependence dominates the local waveform dynamics, making an AR(2)-style approximation reasonable for short ECG segments. Third, because amplitude scales differ across records, per-record z-score normalisation is essential before denoising or classification. Based on these observations, the adopted preprocessing pipeline uses per-record normalisation, stationarity-enhancing filtering or differencing, and R-peak-centered segmentation before feature extraction and sequence modeling.

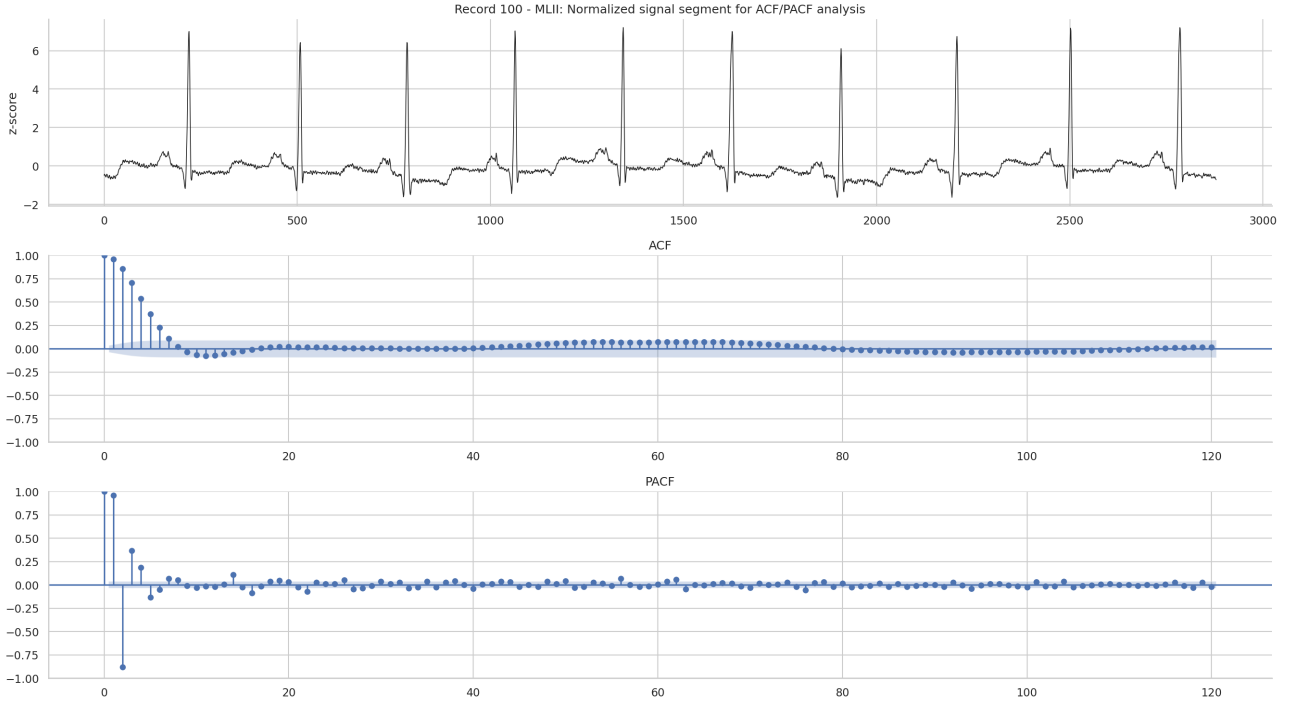


Figure 2: Record 100 – MLII: z-score normalised signal segment with ACF and PACF plots (120 lags).

3.2.2 FPT Stocks Datasets

Price trend and long-term momentum. The closing price series of FPT (`Close_FPT`) is clearly non-stationary, exhibiting a strong upward trend from mid-2017 to early 2025 (Figure 3). Three distinct regimes emerge: (i) a consolidation phase (VND 15–25k, 2017–2020), (ii) a breakout phase (VND 20–60k, 2020–2021) aligned with post-COVID technology revaluation, and (iii) an accelerated bull phase (VND 60–155k, 2022–early 2025) supported by sustained institutional demand. A sharp correction to approximately VND 105k in Q1 2025 suggests a short-term adjustment, potentially driven by profit-taking or market-wide de-risking. Over the full sample, the stock appreciated by roughly 10×, reinforcing its characterization as a structural growth asset.

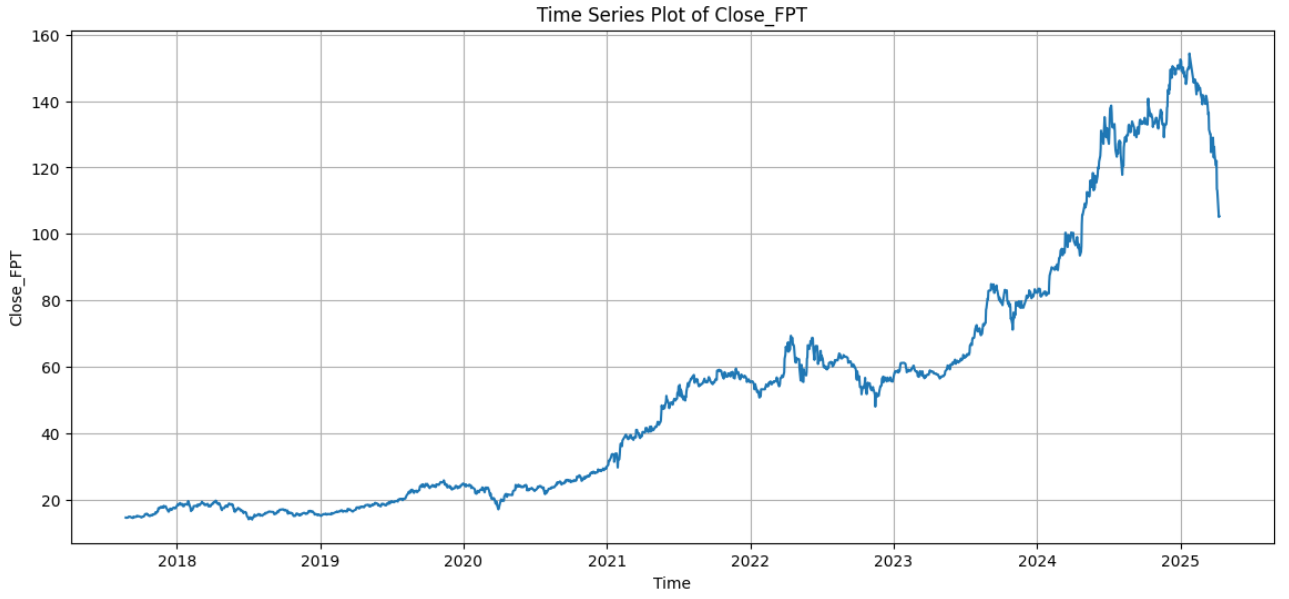


Figure 3: Time series of FPT closing price (`Close_FPT`), 2017–2025.

The 30-day rolling mean closely follows the price series, confirming a persistent upward drift with no mean reversion in levels (Figure 4). In contrast, the rolling standard deviation exhibits clear volatility clustering, rising from near-zero (2017–2020) to moderate levels (2021–2022), and peaking above VND 10 in early 2025. This indicates that risk scales with price, consistent with a momentum-driven regime and increasing late-cycle fragility.

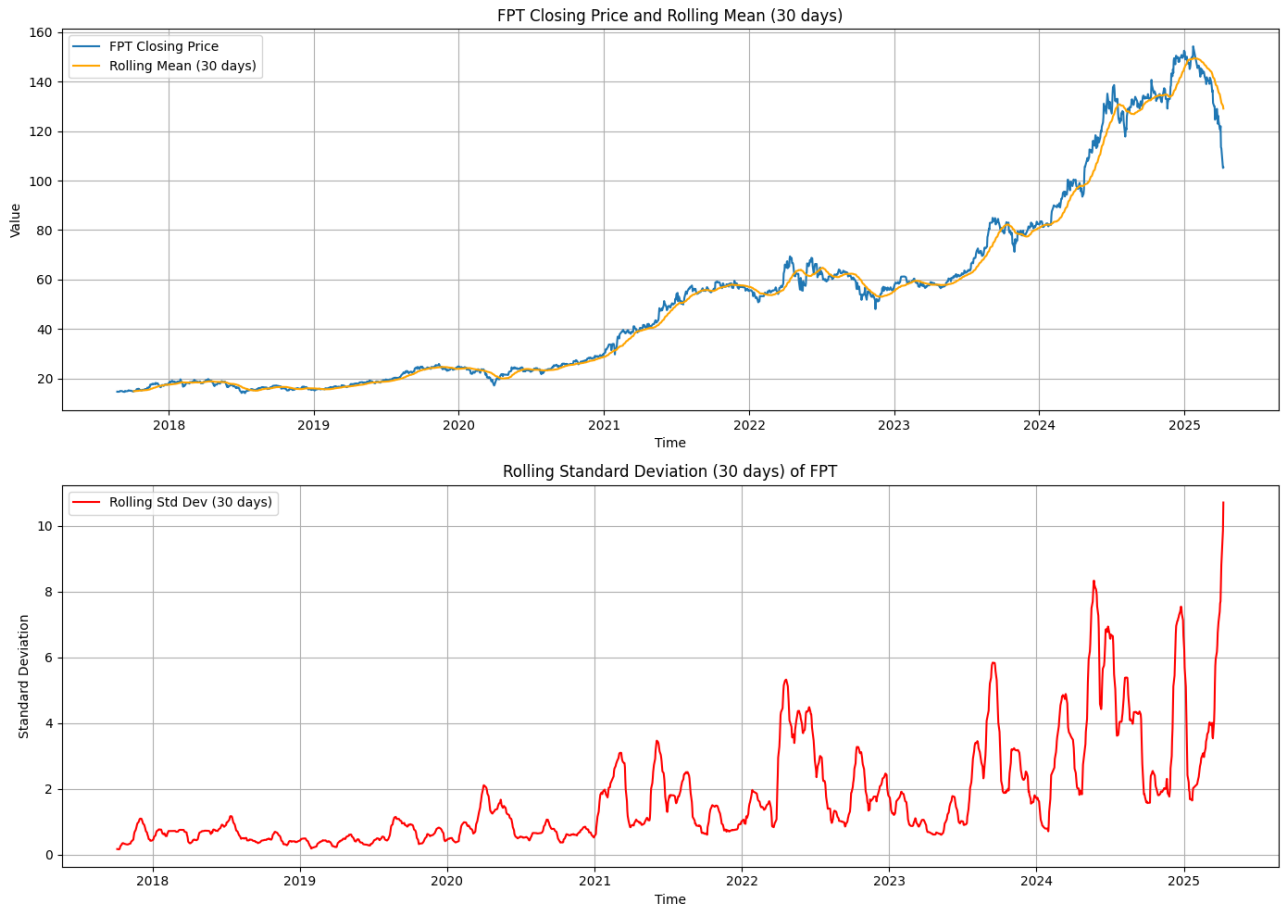


Figure 4: FPT closing price with 30-day rolling mean (upper) and 30-day rolling standard deviation (lower).

Daily returns distribution and tail risk. Daily percentage returns are stationary, centred near zero, and display volatility clustering consistent with ARCH-type dynamics (Figure 5). The return series is broadly symmetric with bounded swings of approximately $\pm 7\%$, but notable negative tail events are visible in March 2020 (COVID shock, $\approx -6.5\%$ single-day), April 2022, and Q1 2025.

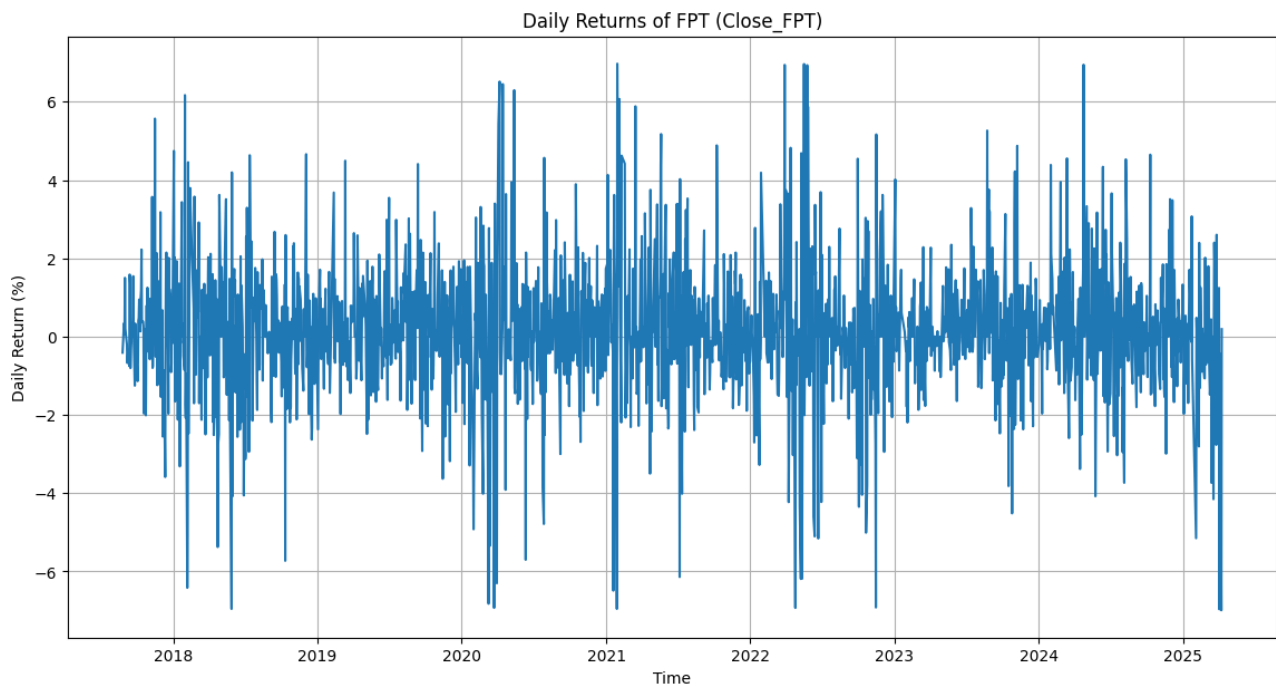


Figure 5: Daily percentage returns of FPT (`Close_FPT`).

Transaction volume dynamics. Trading volume (`Volume_FPT`) is highly right-skewed with persistent spikes, indicating episodic liquidity events rather than a smooth trend (Figure 6). Volume remained subdued and relatively flat from 2017 to mid-2023 (below 5 million shares per session on most days). A structural volume expansion began in late 2023 and intensified through 2024–2025, with multiple sessions exceeding 10–20 million shares—roughly 4–5 $\times$ the historical baseline.

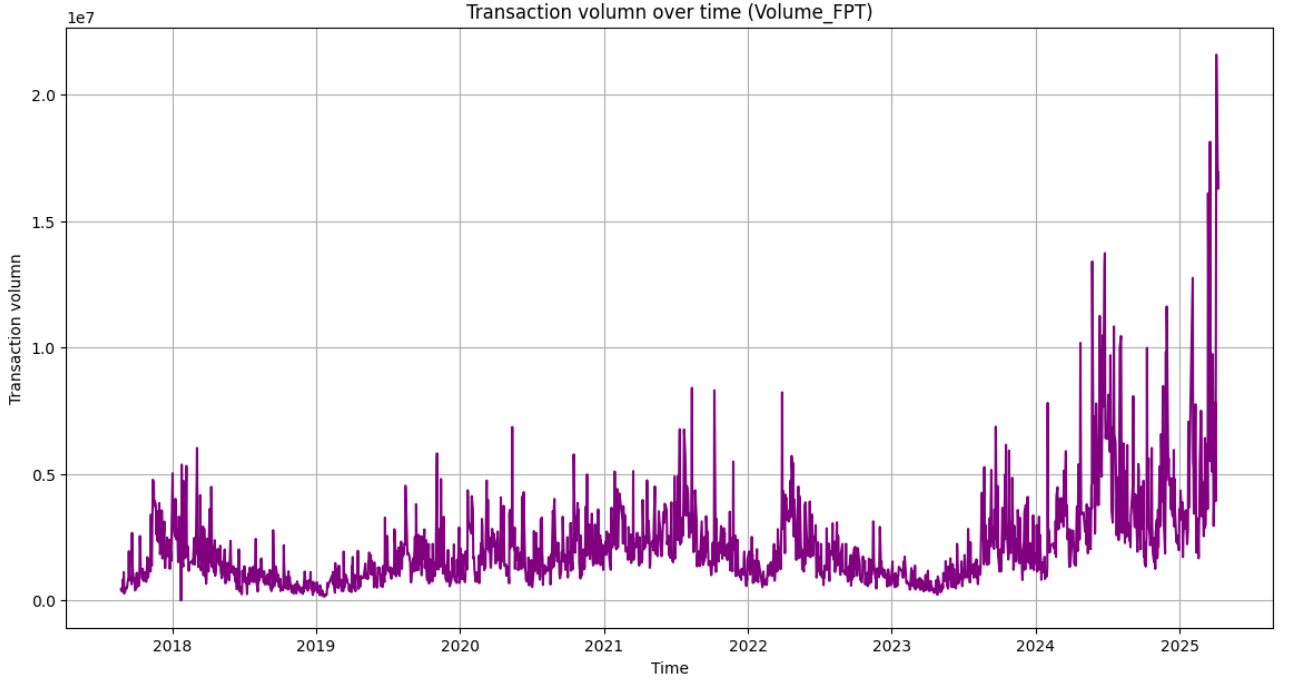


Figure 6: Transaction volume of FPT (Volume_FPT) over time.

The ACF of Close FPT remains near unity across all lags, confirming a non-stationary, $I(1)$ -type process (Figure 7). The PACF shows a single dominant spike at lag 1 with negligible higher-order terms, consistent with an $AR(1)$ /random walk structure. This implies that price dynamics are primarily driven by lag-1 persistence, and first-differencing (e.g., log-returns) is required before any parametric or machine-learning modelling.

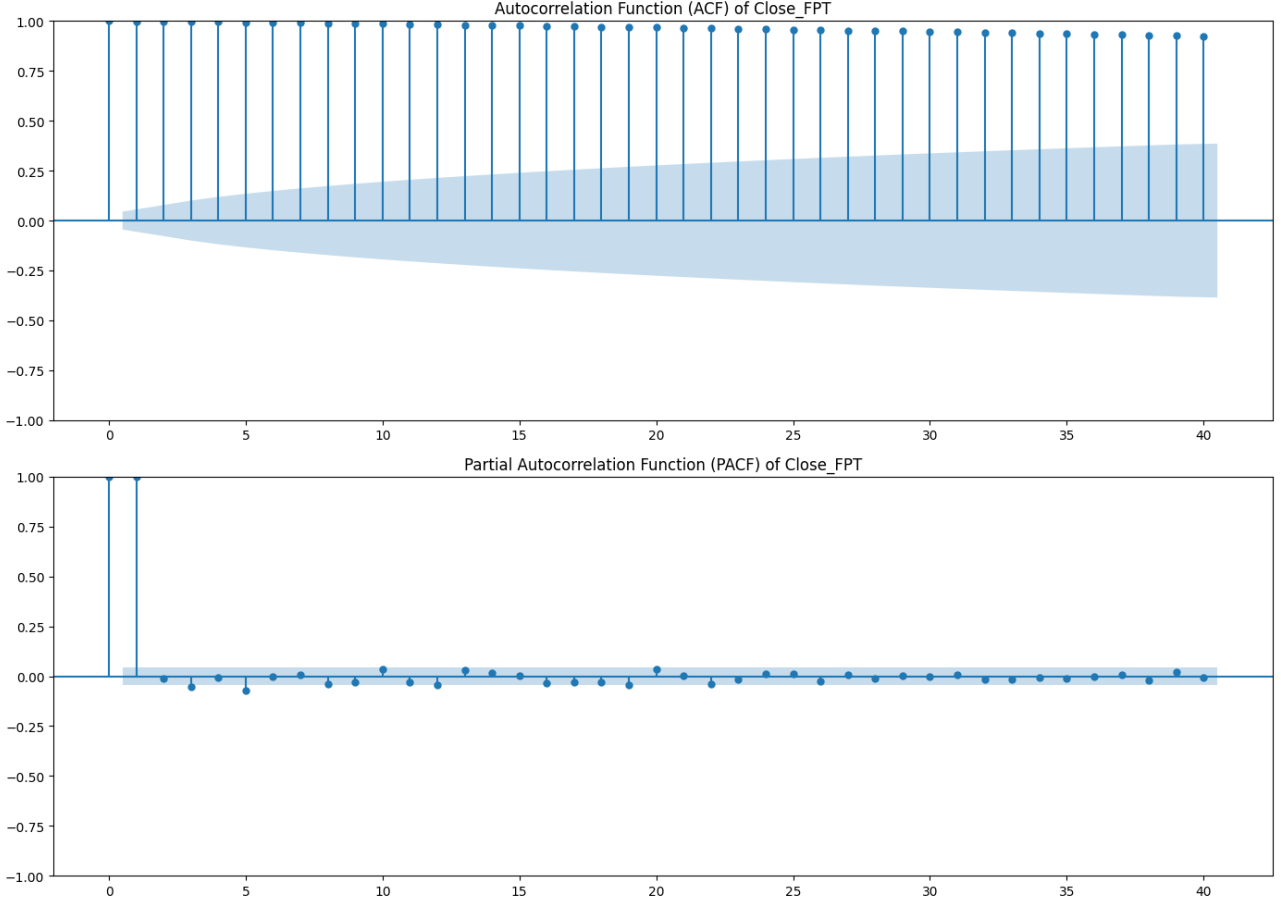


Figure 7: ACF and PACF of `Close_FPT` up to lag 40.

4 Background Preliminaries

This section presents the mathematical foundations of the denoising methods used in the report. The discussion is intentionally focused on concepts needed for the experiments: model assumptions, core equations, denoising mechanism, and practical limitations.

4.1 Filter-based Methods

Filter-based methods operate directly on the observed sequence and aim to suppress local fluctuations while preserving the main temporal structure. They are usually simple to implement and useful as preprocessing steps before statistical modelling or classification.

4.1.1 Median Filter

The median filter is a nonlinear local filter that replaces each observation by the median value in a neighboring window. Let $X = \{x_1, x_2, \dots, x_n\}$ be a discrete time series. For an odd window size $2k + 1$, the local window centered at time t is

$$W_t = \{x_{t-k}, x_{t-k+1}, \dots, x_t, \dots, x_{t+k}\}.$$

After sorting the values inside the window,

$$x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(2k+1)},$$

the filtered output is the central order statistic:

$$y_t = \text{median}(W_t) = x_{(k+1)}.$$

] Boundary points can be handled by reflection, padding, or by applying the filter only to valid indices.

The median filter is closely related to robust estimation. The sample mean minimizes squared deviations,

$$\min_a \sum_i (x_i - a)^2,$$

whereas the median minimizes absolute deviations,

$$\min_a \sum_i |x_i - a|.$$

Because absolute loss grows linearly, the median is much less sensitive to extreme observations. Therefore, isolated spikes are often removed without strongly changing nearby valid samples. This is useful for impulsive noise in ECG recordings and abnormal observations in financial data.

The main parameter is the window radius k . A small window preserves local details but may leave noise; a large window smooths more aggressively but can distort narrow structures such as ECG peaks. Hence, the window size must be chosen according to the expected duration of noise relative to the meaningful signal pattern.

4.1.2 Kalman Filter

The Kalman filter is a recursive estimator for systems described by a state-space model. In the linear-Gaussian case, the hidden state x_t and observation z_t are defined as

$$x_t = Ax_{t-1} + Bu_t + w_t,$$

$$z_t = Hx_t + v_t,$$

where u_t is a control input, $w_t \sim \mathcal{N}(0, Q)$ is process noise, and $v_t \sim \mathcal{N}(0, R)$ is measurement noise. The goal is to estimate the latent clean state x_t from noisy observations z_t .

Each iteration contains a prediction step and an update step. The prediction equations are

$$\hat{x}_{t|t-1} = A\hat{x}_{t-1|t-1} + Bu_t,$$

$$P_{t|t-1} = AP_{t-1|t-1}A^T + Q,$$

where P denotes the estimation covariance. After observing z_t , the Kalman gain is computed as

$$K_t = P_{t|t-1}H^T(H P_{t|t-1}H^T + R)^{-1}.$$

The posterior estimate is then updated by

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(z_t - H\hat{x}_{t|t-1}),$$

$$P_{t|t} = (I - K_tH)P_{t|t-1}.$$

The Kalman gain controls the balance between model prediction and new measurement. If R is large, observations are considered noisy and the filter trusts the model more. If Q is large, the hidden state is allowed to vary more quickly and the filter follows the observations more closely. This creates a bias-variance trade-off: strong smoothing reduces variance but may lag behind true changes, while weak smoothing preserves rapid variations but may retain noise.

The standard Kalman filter is optimal under linear dynamics, Gaussian noise, and correctly specified covariance matrices. It is computationally efficient for one-dimensional sequences, with $\mathcal{O}(T)$ time complexity over a series of length T . However, performance may degrade for impulsive noise, nonlinear dynamics, or time-varying volatility unless extensions such as the Extended Kalman Filter or Unscented Kalman Filter are used.

4.1.3 Spectral Gating

Spectral gating reduces noise by transforming a signal into the frequency domain, attenuating frequency components that are likely to represent noise, and reconstructing the signal. Suppose the observed signal is

$$x_t = s_t + n_t,$$

where s_t is the clean component and n_t is noise. The discrete Fourier transform (DFT) of a sequence $x_0, \dots, x_{N-1}$ is

$$X_k = \sum_{t=0}^{N-1} x_t e^{-j2\pi kt/N}, \quad k = 0, 1, \dots, N-1.$$

Each coefficient can be represented by magnitude and phase:

$$X_k = |X_k|e^{j\phi_k}.$$

A spectral mask M_k is then applied:

$$\tilde{X}_k = M_k X_k, \quad 0 \leq M_k \leq 1.$$

The denoised signal is obtained using the inverse DFT:

$$\tilde{x}_t = \frac{1}{N} \sum_{k=0}^{N-1} \tilde{X}_k e^{j2\pi kt/N}.$$

For nonstationary signals, the same idea is usually applied with the short-time Fourier transform (STFT), which analyzes local frequency content:

$$X(m, \omega) = \sum_n x[n] w[n - m] e^{-j\omega n},$$

where $w[\cdot]$ is a window function. A threshold is estimated from a noise profile, and bins below the threshold are attenuated. This method is useful when noise and signal occupy different spectral regions. Its limitation is that meaningful signal components may be removed if their frequency overlaps with noise. For ECG, excessive frequency suppression can distort QRS morphology; for financial data, it may remove abrupt but informative movements.

4.2 Seasonal and Trend Decomposition Using Loess (STL)

Seasonal and Trend Decomposition Using Loess (STL) decomposes a time series into three additive components:

$$Y_t = T_t + S_t + R_t,$$

where T_t is the long-term trend, S_t represents repeated seasonal behavior, and R_t is the irregular remainder. STL estimates these components using LOESS smoothing, which fits local weighted regressions without assuming a fixed global form for the signal.

For a target point x_0 , LOESS assigns larger weights to observations located near x_0 . A commonly used tricube weight is

$$w_i = \left(1 - \left| \frac{x_i - x_0}{d} \right|^3 \right)^3,$$

where d is the largest distance within the selected neighborhood. A local polynomial, usually linear, is then estimated by weighted least squares:

$$\min_{\beta_0, \beta_1} \sum_i w_i (y_i - \beta_0 - \beta_1 x_i)^2.$$

The fitted value at x_0 becomes the smoothed estimate. By repeating this process over all observations, LOESS captures gradual nonlinear variation in both seasonal and trend components.

Inner loop: seasonal and trend extraction. STL uses an inner loop to iteratively separate the seasonal component from the trend component. Let n_p be the seasonal period, such as $n_p = 12$ for monthly data with yearly seasonality. Starting from an initial trend estimate,

commonly $T_t^{(0)} = 0$, each inner iteration updates the components as follows:

$$D_t^{(k+1)} = Y_t - T_t^{(k)},$$

where $D_t^{(k+1)}$ is the detrended series. The observations in $D_t^{(k+1)}$ are then divided into n_p cycle-subseries according to their seasonal positions. For example, monthly observations are separated into January, February, ..., December subseries. Each subseries is smoothed by LOESS with seasonal window n_s , producing a temporary seasonal sequence $C_t^{(k+1)}$.

Because $C_t^{(k+1)}$ may still contain low-frequency movement belonging to the trend, STL applies a low-pass filter:

$$L_t^{(k+1)} = \text{LOESS}_{n_l} \left(\text{MA}_3 \left(\text{MA}_{n_p} \left(\text{MA}_{n_p} \left(C_t^{(k+1)} \right) \right) \right) \right),$$

where n_l is the low-pass smoothing window. The seasonal component is obtained by removing this low-frequency part:

$$S_t^{(k+1)} = C_t^{(k+1)} - L_t^{(k+1)}.$$

Next, the original series is deseasonalized and smoothed to update the trend:

$$A_t^{(k+1)} = Y_t - S_t^{(k+1)}, \quad T_t^{(k+1)} = \text{LOESS}_{n_t} \left(A_t^{(k+1)} \right),$$

where n_t is the trend smoothing window. These steps are repeated for n_i inner iterations. The residual after the final update is

$$R_t = Y_t - T_t - S_t.$$

Thus, the inner loop alternates between estimating seasonality from a detrended series and estimating trend from a deseasonalized series, reducing leakage between the two components.

Outer loop: robust reweighting. When abnormal observations or transient spikes are present, standard smoothing may allow them to distort the estimated trend and seasonal components. Robust STL therefore introduces an outer loop. After one complete inner-loop run, residuals are used to compute robustness weights. Using the median absolute deviation,

$$\text{MAD} = \text{median}(|R_t|),$$

the bisquare robustness weight for observation t is

$$\rho_t = \begin{cases} \left(1 - \left(\frac{R_t}{6\text{MAD}} \right)^2 \right)^2, & |R_t| < 6\text{MAD}, \\ 0, & \text{otherwise.} \end{cases}$$

Observations with large residuals receive smaller weights or are completely ignored. In the next outer iteration, the LOESS weights are adjusted by

$$\tilde{w}_{i,t} = w_{i,t}\rho_i,$$

and the entire inner-loop procedure is repeated. Therefore, the outer loop does not estimate an additional component; instead, it repeatedly refines S_t and T_t while reducing the influence of outliers. After n_o outer iterations, the final trend, seasonal, and residual components are obtained.

Parameters and applicability. The main STL parameters are the seasonal period n_p , seasonal smoothing window n_s , trend smoothing window n_t , low-pass window n_l , number of inner iterations n_i , and number of outer iterations n_o . Larger smoothing windows generate smoother components but may fail to capture rapid changes. Increasing n_i improves separation between trend and seasonality, whereas increasing n_o improves robustness at additional computational cost.

STL is interpretable and effective when a signal contains a relatively stable seasonal structure. However, it is less appropriate when periodicity is weak, when strong structural breaks occur, or when the objective is to model complex nonlinear dependence rather than to decompose the observed signal into interpretable components.

4.3 Auto Regressive Integrated Moving Average (ARIMA)

ARIMA is a model-based approach for representing autocorrelation in a univariate time series. It combines autoregressive terms, differencing, and moving-average terms. Let B denote the backshift operator, $By_t = y_{t-1}$. An ARIMA(p, d, q) model is written as

$$\phi(B)(1 - B)^d y_t = c + \theta(B)\varepsilon_t,$$

where

$$\phi(B) = 1 - \phi_1 B - \cdots - \phi_p B^p,$$

$$\theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q.$$

The parameter p is the autoregressive order, d is the number of differences used to achieve stationarity, and q is the moving-average order.

The autoregressive component models dependence on previous values:

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t.$$

The moving-average component models dependence on previous shocks:

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}.$$

Differencing removes nonstationary trend-like behavior:

$$\nabla y_t = y_t - y_{t-1} = (1 - B)y_t.$$

For denoising, the fitted ARIMA values are interpreted as the structured predictable component, while the residuals are treated as noise:

$$y_t = \hat{y}_t + \hat{\varepsilon}_t.$$

The residual sequence should ideally behave like white noise. Residual autocorrelation can be checked using the autocorrelation function or Ljung–Box test. Model order is commonly chosen by information criteria:

$$\text{AIC} = 2k - 2 \log L,$$

$$\text{BIC} = k \log n - 2 \log L,$$

where k is the number of parameters, n is the sample size, and L is the likelihood.

ARIMA is transparent and useful when the signal contains linear autocorrelation. It is appropriate for both trend extraction and feature generation. However, it assumes that the differenced process is stationary and primarily linear. It may therefore be insufficient for strongly nonlinear ECG morphology or financial regimes with abrupt structural changes. Seasonal behavior can be handled using SARIMA(p, d, q)(P, D, Q) $_s$.

4.4 Long Short-Term Memory (LSTM)

LSTM is a recurrent neural network architecture designed to learn long-range temporal dependencies. Unlike a standard recurrent network, an LSTM uses a memory cell and gates to control information flow. Given input x_t , previous hidden state h_{t-1} , and previous cell state c_{t-1} , the gates are

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o),$$

where f_t , i_t , and o_t are the forget, input, and output gates. The candidate memory is

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c).$$

The cell and hidden states are updated by

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

$$h_t = o_t \odot \tanh(c_t).$$

The sigmoid function $\sigma(\cdot)$ restricts gate values to $[0, 1]$, allowing the network to learn what information should be retained or discarded.

For denoising, LSTM is often used in an autoencoder structure. The encoder maps a noisy sequence into a latent representation, and the decoder reconstructs a cleaner sequence. If x_t is the input and $\hat{x}_t$ is the reconstructed output, a common objective is mean squared error:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{T} \sum_{t=1}^T (x_t - \hat{x}_t)^2.$$

When clean targets are unavailable, the model can be trained to reconstruct smoothed signals or to learn stable representations from corrupted inputs. Regularization such as dropout, early stopping, and weight decay is important to prevent the model from simply memorizing noise.

Compared with ARIMA and STL, LSTM can capture nonlinear and multivariate temporal patterns. This flexibility is useful for ECG morphology and sequential financial features. However, LSTM models require more data, careful tuning, and provide weaker interpretability. In this report, LSTM-based denoising is used as a learned counterpart to classical statistical denoising methods.

5 Experiment Setup

5.1 MIT-BIH Database

This study employs a patient wise split on the MIT-BIH Arrhythmia Database, using 27 records for training (DS1) and 9 disjoint records for testing (DS2). The original 15-class annotations are mapped into a clinically relevant 3-class problem: *Normal* (N, L, R, e, j), *Atrial* (A, a, J, S), and *Other* (V, E, F, /, f, Q).

5.1.1 Backbone Model Architecture

The core classification model, named **MultiInput_BiLSTM_Attention**, is a multi-branch architecture designed to effectively fuse morphological, temporal rhythm, and handcrafted features. Figure 8 illustrates the overall structure of this backbone.

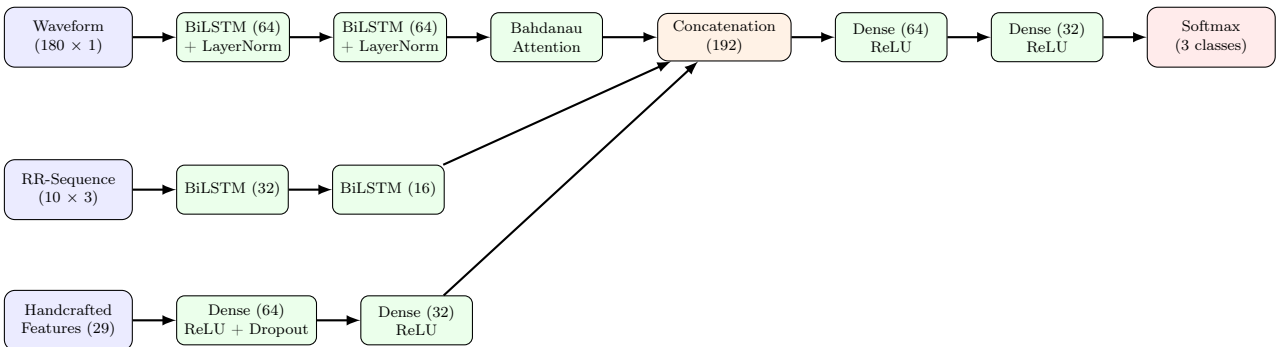


Figure 8: Multi-input Backbone Architecture (**MultiInput_BiLSTM_Attention**).

The backbone consists of three parallel branches:

- **Waveform Branch:** Accepts a z-score normalized 180-sample ECG segment. It contains two stacked Bidirectional LSTM layers (64 units each) with Layer Normalization and dropout rate 0.3, followed by a Bahdanau additive attention layer.
- **RR-Sequence Branch:** Takes a sequence of 10 consecutive beats (each with 3 RR features: RR interval, BPM, RR z-score). This branch uses two Bidirectional LSTM layers (32 and 16 units).
- **Handcrafted Feature Branch:** A 29-dimensional vector combining morphological statistics, wavelet features, and RR features, processed by two dense layers (64 and 32 units) with ReLU activation and dropout.

The outputs of the three branches are concatenated (192 dimensions) and fed into two dense layers ($64 \rightarrow 32$) before the final softmax output layer. L2 regularization ($\lambda = 10^{-4}$) is applied to LSTM and dense layers. The model is trained with Adam optimizer (learning rate = 10^{-3}) and categorical cross-entropy loss.

5.1.2 Overall Pipeline

The complete experimental pipeline is shown in Figure 11. Raw ECG signals first undergo baseline wander removal using a two-stage median filter (kernel sizes 71 and 215), followed by a 4th-order Butterworth bandpass filter (0.5–40 Hz). Each heartbeat is segmented into 180 samples centered at the R-peak and z-score normalized per beat.

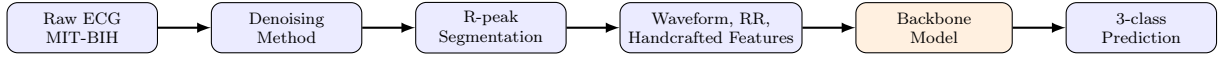


Figure 9: Overall experimental pipeline for the MIT-BIH classification experiment. Different denoising methods are applied before segmentation and feature extraction, while the same backbone model is used for classification.

For every beat, we extract:

- The normalized waveform segment,
- RR-interval features (RR, BPM, RR_z),
- Handcrafted features (morphological + wavelet decomposition using db1 wavelet at level 3).

A patient-specific rhythm sequence of 10 consecutive beats is constructed for the RR-sequence branch (with appropriate padding at the start of each record). To handle severe class imbalance, SMOTE is applied on the combined feature space, and class weights are additionally used during training. All models use early stopping (patience=7) with maximum 40 epochs and learning rate reduction on plateau.

5.1.3 Denoising and Trend Extraction Modules

Median Filter Denoiser The median filter is used as an early-stage preprocessing module in the MIT-BIH ECG classification pipeline. It is applied to the continuous ECG signal before heartbeat segmentation to reduce baseline wander and impulsive artifacts while preserving important waveform morphology such as the QRS complex. To evaluate its effectiveness, we compare a baseline setting using bandpass filtering only with a second setting using two-stage median filtering followed by the same bandpass filter. All downstream components, including segmentation, feature extraction, patient-wise train/test split, classifier architecture, and evaluation metrics, are kept unchanged, so any performance difference reflects the contribution of median filtering.

Kalman Filtering The Kalman filter is implemented as a beat-level signal denoiser in the MIT-BIH ECG pipeline, applied to each segmented heartbeat window prior to feature extraction and classification. An initial investigation applying Kalman filtering to MIT-BIH heartbeat classification with a Logistic Regression backbone revealed fundamental incompatibility between the filter’s core assumptions and ECG signal characteristics, leading to its exclusion from the final experimental pipeline.

The incompatibility arises because the filter assumes the underlying signal follows a random walk with white Gaussian noise – a model appropriate for slowly drifting processes. ECG beats, however, are morphologically structured biological oscillations driven by a cardiac cycle, not a random walk, and their noise is neither white nor Gaussian. As a result, the constant Kalman model behaves as a low-pass filter that suppresses the QRS band (40–150 Hz), precisely the frequency range encoding the morphological differences between normal and abnormal beats. This structural mismatch is not resolvable through parameter tuning – it is inherent to the model class. Consequently, Kalman filtering destroys the discriminative signal rather than isolating noise, reducing class separability and degrading classification performance compared to using raw beats directly.

Despite this incompatibility, Kalman filtering holds genuine promise for ECG baseline wander removal on continuous recordings. Baseline wander is a slow-drifting, low-frequency artifact (0.05–0.5 Hz) caused by breathing, patient movement, and electrode drift. Critically, it behaves as a random walk – the baseline level at the next sample is best predicted by its current value plus a small stochastic increment – precisely the process the constant Kalman model was designed to track, making it both theoretically valid and practically advantageous over z-normalization for this application.

Specifically, Kalman baseline wander removal offers four concrete advantages over z-normalization on continuous ECG recordings:

- **Intra-window drift correction.** A breathing cycle of approximately 0.3 Hz completes roughly 20% of a full oscillation within a 0.7-second beat window, producing a visible slope. Z-normalization subtracts a single global mean, which is pulled upward by the

drifting tail, artificially suppressing R-peak amplitude and distorting T-wave morphology. Kalman filtering tracks the baseline as a time-varying trajectory, subtracting a different estimated value at every sample and preserving beat morphology under drift.

- **Preservation of absolute amplitude.** Z-normalization forces every beat to unit variance, destroying clinically meaningful inter-beat amplitude differences – a high-voltage QRS indicative of left ventricular hypertrophy and a low-voltage QRS associated with pericardial effusion become indistinguishable after scaling. Kalman baseline removal subtracts only the estimated drift without rescaling, preserving absolute amplitude information across beats.
- **Signal energy preservation.** Dividing by standard deviation forces all beats to the same energy level, making a noisy beat and a clean beat indistinguishable in scale. Kalman filtering leaves the signal variance intact, keeping noise level differences between beats visible and interpretable.
- **Causal real-time operation.** Z-normalization is acausal and offline – it requires the full beat window before computation can begin. Kalman filtering is a causal recursive estimator that at each sample uses only past observations, making it suitable for real-time clinical monitors, implantable devices, and wearable systems where latency constraints preclude buffering entire windows.

Nevertheless, since the MIT-BIH pipeline operates on pre-segmented, z-normalized beat windows in an offline batch setting, none of these advantages materialise in practice: intra-window drift is negligible over a 0.7-second window, absolute amplitude is irrelevant to binary morphology classification, signal energy normalisation is a deliberate preprocessing choice, and real-time processing is not required. Under these conditions, z-normalization is sufficient and Kalman baseline wander removal introduces unnecessary complexity without measurable benefit.

Spectral Gating Denoiser A spectral gating module is implemented as a frequency-domain denoising component for ECG heartbeat classification. Unlike ARIMA, which models each beat as a short autoregressive time series, and unlike LSTM-DAE, which learns a nonlinear reconstruction function from corrupted beats, spectral gating suppresses noise by attenuating noise-dominated time–frequency components in the short-time Fourier transform (STFT) domain.

Let $x[n]$ denote a preprocessed ECG signal after median baseline removal and bandpass filtering. The STFT representation of the signal is defined as

$$X(m, \omega) = \sum_n x[n] w[n - m] e^{-j\omega n}, \quad (1)$$

where $w[\cdot]$ is the analysis window, m denotes the local time frame, and ω denotes the angular frequency.

A soft spectral mask $M(m, \omega)$ is then estimated from the local noise floor and applied to the STFT coefficients:

$$\tilde{X}(m, \omega) = M(m, \omega)X(m, \omega), \quad 0 \leq M(m, \omega) \leq 1. \quad (2)$$

Finally, the denoised signal is reconstructed using the inverse STFT:

$$\tilde{x}[n] = \text{iSTFT}(\tilde{X}(m, \omega)). \quad (3)$$

In the ECG experiment, spectral gating is evaluated in two complementary settings. The first is beat-level spectral gating, where each 180-sample segmented heartbeat is denoised independently after R-peak segmentation. This setting is directly comparable with the ARIMA and LSTM-DAE phases because the denoising operation is applied to the waveform input of each beat, while RR-sequence features and handcrafted features remain unchanged.

The second is record-level spectral gating, where the entire preprocessed ECG record is denoised before R-peak segmentation. This setting is closer to the STL phase because the denoising operation is performed on the continuous signal before beat-level representation learning.

The complete record-level spectral-gating workflow is therefore

$$\begin{aligned} \text{Raw ECG} &\rightarrow \text{Median} + \text{Bandpass Filter} \\ &\rightarrow \text{Spectral Gating} \\ &\rightarrow \text{R-peak Segmentation} \\ &\rightarrow \text{Waveform, RR, Handcrafted Features} \\ &\rightarrow \text{MultiInput_BiLSTM_Attention.} \end{aligned} \quad (4)$$

For the beat-level variant, the spectral gating step is applied immediately after R-peak segmentation rather than before it:

$$\begin{aligned} \text{Raw ECG} &\rightarrow \text{Median} + \text{Bandpass Filter} \\ &\rightarrow \text{R-peak Segmentation} \\ &\rightarrow \text{Beat-level Spectral Gating} \\ &\rightarrow \text{Waveform, RR, Handcrafted Features} \\ &\rightarrow \text{MultiInput_BiLSTM_Attention.} \end{aligned} \quad (5)$$

This design keeps the downstream classifier, label mapping, feature structure, patient-wise split, and training protocol unchanged. Consequently, any performance difference can be attributed primarily to the spectral denoising strategy rather than to changes in the classifier architecture. Spectral gating is expected to be beneficial when noise occupies time–frequency regions that are separable from the main ECG morphology. However, clinically relevant ECG components such as the QRS complex also contain high-frequency information. Therefore, a soft attenuation mask is preferred over hard thresholding to reduce noise while preserving

discriminative waveform morphology.

STL Denoiser An STL-based denoising module is implemented as a non-parametric trend and residual extraction method for ECG signals. Unlike per-beat denoising methods, STL requires information across repeated cycles in order to estimate meaningful seasonal and trend components. Therefore, applying STL directly to a 180-sample beat segment is not appropriate: at the MIT-BIH sampling rate of 360 Hz, 180 samples correspond to only 0.5 seconds, which is too short to contain sufficient cardiac cycles for reliable decomposition.

To address this limitation, STL is applied at the record level before beat segmentation. The raw ECG signal is first processed by median filtering and bandpass filtering, after which STL decomposition is performed on the longer record-level signal. The residual component is then used as the denoised ECG representation. R-peak segmentation is subsequently applied to this STL-denoised signal, producing 180-sample beat waveforms for the same downstream `MultiInput_BiLSTM_Attention` backbone.

The complete STL-based phase follows this workflow: raw ECG preprocessing, record-level STL decomposition, denoised-signal reconstruction, R-peak beat segmentation, RR-sequence construction, handcrafted morphological/wavelet feature extraction, feature balancing, and multi-input classification. This design preserves the same classifier and feature structure as the baseline while introducing STL as a controlled record-level denoising step before beat-level representation learning.

ARIMA Denoiser An ARIMA-based denoising module is implemented as a statistical alternative to the LSTM-DAE. Each z-score normalized ECG beat is treated as a short univariate time series of length 180. The ARIMA model captures the structured temporal dependence of the beat morphology, including the local dynamics around the P-wave, QRS complex, and T-wave. Its fitted values are interpreted as the denoised physiological component, while the residuals are treated as random noise caused by baseline drift, EMG interference, or motion artifacts.

For each beat, an ARIMA(2, 1, 2) model is fitted independently. The first-order differencing term helps suppress low-frequency drift, while the autoregressive and moving-average terms model local waveform continuity. After fitting, the reconstructed fitted sequence is used as the denoised waveform input to the waveform branch of the same `MultiInput_BiLSTM_Attention` backbone. RR-interval features and handcrafted morphological/wavelet features remain unchanged, so the only controlled difference from the baseline is the waveform denoising mechanism.

The complete ARIMA-based phase follows this workflow: raw ECG preprocessing, beat segmentation, per-beat z-score normalization, ARIMA denoising, RR-sequence construction, feature balancing, and multi-input classification. This design makes ARIMA directly comparable with the LSTM module: ARIMA provides an interpretable model-based denoising layer, whereas LSTM provides a learned nonlinear denoising layer. Comparing both phases helps evaluate whether classical statistical smoothing or learned sequence reconstruction provides

cleaner and more discriminative ECG representations.

LSTM Denoiser An LSTM-based Denoising Autoencoder is implemented as a modular preprocessing component to remove common ECG artifacts (baseline wander, EMG noise, motion artifacts). The model architecture is as follows:

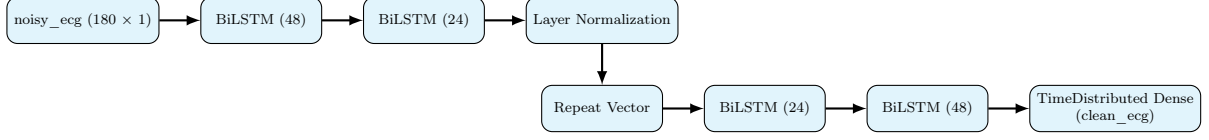


Figure 10: LSTM Denoising Autoencoder Architecture.

The DAE is trained by adding Gaussian noise (factor = 0.4) to clean beats and reconstructing the original signal using MSE loss. After training, the model is used to denoise both training and testing waveforms before they are fed into the backbone classifier in Phase 3. This design allows easy substitution with other denoising methods in future work.

5.2 FPT Stocks Database

5.2.1 Dataset and Labeling Strategy

Daily OHLCV data is resampled to weekly frequency using W-FRI (Friday close as the weekly close). A week is labeled as:

- **Up** if next week's return $> +2.0\%$,
- **Down** if next week's return $< -2.0\%$,
- **Sideways** otherwise $\rightarrow$ dropped from the dataset.

This `SIGNAL_THR = 2.0%` threshold removes noisy weeks and allows the model to focus on clear directional movements.

5.2.2 Backbone Model Architecture

We evaluate two main families of models: tree-based ensemble and deep sequential models.

Tree-based Ensemble LightGBM + RandomForest ensemble (weighted 0.6 : 0.4). LightGBM uses `max_depth=4`, `num_leaves=15`; RandomForest uses `max_depth=6`. Both are re-trained on the expanding training window each fold.

BiLSTM + Attention Backbone The deep learning backbone (`BiLSTM_Attention`) has the following architecture:

The model takes a sequence of the last 12 weeks (`SEQ_LEN = 12`). It consists of one Bidirectional LSTM layer (48 units) with dropout 0.3 and recurrent dropout 0.1, followed by Layer Normalization, Bahdanau attention, and a dense classification head.

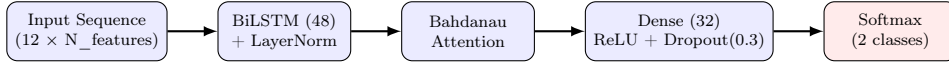


Figure 11: BiLSTM + Attention Backbone Architecture.

5.2.3 Overall Pipeline

Raw daily data is converted to weekly OHLCV. After labeling and dropping sideways weeks, approximately 35 handcrafted technical features are engineered, including multi-timeframe returns, momentum, EMA crossovers, MACD, RSI, Bollinger Bands, volume ratios, streak counts, and calendar features. All features are leakage-free (calculated only up to week $t - 1$).

A walk-forward expanding window scheme is used with 5 folds. The first 75% of data serves as the initial training set; the remaining data is evaluated in successive validation folds (~ 8 – 10 weeks per fold). SMOTE is not used due to the sequential nature and walk-forward constraint; instead, class weights are applied during training.

5.2.4 Denoising and Trend Extraction Modules

To improve signal-to-noise ratio in weekly financial time series (similar to the preprocessing steps applied on the MIT-BIH Arrhythmia Database), we implement and compare six complementary denoising and trend-extraction mechanisms. These modules are used as optional preprocessing stages before feeding data into the classifiers.

Median Filtering A median filtering module is applied to the weekly closing-price series as a simple and robust noise-reduction technique. Unlike moving-average smoothing, this filter preserves local trend structures while suppressing isolated spikes and short-term fluctuations that frequently occur in financial markets due to temporary shocks or irregular trading activity. For each time step, the filtered value is computed as the median of observations within a sliding historical window, ensuring a causal implementation that avoids future information leakage. Technical indicators are subsequently recomputed from the filtered series so that downstream models operate on the denoised signal. This preprocessing step is expected to improve the stability of feature extraction while retaining the underlying market trend and directional movements.

Kalman Filtering For the FPT weekly stock price classification task, the Adaptive Kalman Filter is applied as a pure preprocessing step to the log-transformed weekly close price series – all downstream technical indicators are rebuilt on the Kalman-smoothed series, isolating the denoising contribution from any feature engineering effect. By smoothing microstructure noise from the price series before feature engineering, the filter is expected to produce cleaner technical indicators – reducing spurious signals in momentum, volatility, and trend features – and thereby improve the discriminability of the downstream classification task. Log transformation is applied before filtering for two reasons: log prices are well-approximated by a random walk, satisfying the filter’s state transition assumption, and operating in log space ensures scale-

invariance across price levels, so that the filter behaves consistently regardless of the absolute price magnitude.

The adaptive variant is chosen over classic Kalman because stock volatility is heteroskedastic – noise levels shift across market regimes – making a fixed measurement noise R inappropriate. By updating R online from the innovation sequence via an exponential moving average, the filter self-adjusts to changing market conditions without requiring manual recalibration. The filter adopts a constant-acceleration kinematic state model with state vector comprising price, velocity, and acceleration in log space – velocity corresponds to instantaneous log return and acceleration captures momentum change – reflecting the empirical tendency of weekly price trends to persist over consecutive periods. Process noise Q and measurement noise R are calibrated from the training window of each walk-forward fold using return variance and second-difference variance respectively, ensuring no future leakage into parameter estimation.

The pipeline is evaluated on two backbone classifiers, both receiving only the engineered technical indicators rebuilt on the Kalman-smoothed close price – no features are extracted directly from the filter internals. The Tree Ensemble receives unscaled features, while the BiLSTM with Bahdanau Self-Attention receives standardised features over 12-week input windows. Both backbones follow the same walk-forward scheme and evaluation protocol described in the overall pipeline section above.

Spectral Gating Feature Module A spectral gating feature module is implemented as a frequency-domain noise reduction and trend-extraction method for the weekly FPT stock series. While ARIMA captures linear autocorrelation, LSTM-DAE learns nonlinear denoised price representations, and STL decomposes the series into trend, seasonal, and residual components, spectral gating focuses on the distribution of signal energy across frequency bands.

For financial data, the module is applied in a strictly causal walk-forward manner. At each week t , only historical observations up to week t are used to estimate the spectral representation. This prevents future leakage and preserves the validity of the expanding-window evaluation scheme. Given a historical window of closing prices or log-returns, the STFT is computed and a soft spectral mask is applied to attenuate low-energy or noise-dominated components. The inverse STFT then produces a denoised local price or return trajectory.

From this denoised representation, we extract auxiliary spectral features:

- **spectral_denoised**: denoised closing-price or return component.
- **spectral_resid**: difference between the original series and the denoised component.
- **spectral_resid_abs**: absolute residual magnitude, representing irregular fluctuation intensity.
- **spectral_trend_grad**: gradient of the denoised component, indicating directional momentum.
- **spectral_ret**: return computed from the denoised component.

- `spectral_entropy`: concentration or dispersion of frequency-domain energy.
- `spectral_dominant_frequency`: dominant periodic component within the historical window.

These spectral features are concatenated with the base weekly technical features before being passed to the downstream classifiers. For the tree-based ensemble, they provide additional nonlinear descriptors of trend smoothness and irregular noise intensity. For the BiLSTM + Attention backbone, they enrich the weekly input sequence with causal frequency-domain information while preserving the same sequence length and walk-forward training protocol.

The spectral module is therefore evaluated under the same two model families as the other denoising methods:

- Tree Ensemble + Spectral Features
- BiLSTM + Attention + Spectral Features

STL Trend Extraction Module We employ an STL-based trend extraction module to decompose the weekly closing-price series into interpretable time-series components. The decomposition produces a smooth trend component, a seasonal component, and a residual component, which are then used to construct auxiliary features for downstream classification models.

From the STL decomposition, we extract seven auxiliary features:

- `stl_trend`: Smoothed underlying trend component.
- `stl_seasonal`: Seasonal component capturing repeated weekly patterns.
- `stl_resid`: Residual component after removing trend and seasonality.
- `stl_resid_abs`: Absolute residual magnitude, representing irregular fluctuation intensity.
- `stl_detrended`: Closing price after removing the STL trend component.
- `stl_trend_grad`: Gradient of the STL trend, indicating trend momentum.
- `stl_ret`: Percentage return computed from the STL trend component.

This module provides a flexible non-parametric decomposition of the weekly closing-price series. The extracted STL features summarize trend direction, seasonal behavior, and irregular deviations, thereby enriching the feature set used by the downstream classification models.

LSTM Denoising Autoencoder Similar to the successful LSTM Denoising Autoencoder used in the MIT-BIH ECG arrhythmia classification pipeline, we implement a deep learning-based denoising module specifically adapted for financial time series.

The LSTM-DAE architecture consists of:

- **Encoder:** Two Bidirectional LSTM layers (32 and 16 units) with Layer Normalization.
- **Bottleneck:** Latent representation repeated across the time dimension.
- **Decoder:** Two Bidirectional LSTM layers (16 and 32 units) followed by a TimeDistributed Dense layer.

The model is trained by injecting Gaussian noise (factor = 0.15) into scaled weekly price sequences and reconstructing the clean original sequences using Mean Squared Error (MSE) loss. After training, the DAE is used to generate denoised price representations in a walk-forward fashion.

This denoised price sequence is then concatenated with the handcrafted technical features before being passed to the BiLSTM + Attention backbone.

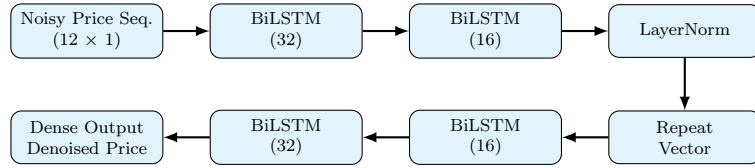


Figure 12: LSTM Denoising Autoencoder Architecture (adapted from MIT-BIH ECG pipeline).

ARIMA Trend Extraction Module We first employ a classical statistical approach using an ARIMA(1,1,1) model fitted in a strictly causal walk-forward manner. For each week t , the model is trained solely on the previous 104 weeks (≈ 2 years) of closing prices to avoid any future leakage.

From the fitted ARIMA model, we extract five auxiliary features:

- **arma_fitted:** Smoothed underlying trend component.
- **arma_resid:** Residuals (noise component).
- **arma_resid_abs:** Magnitude of residuals, capturing noise intensity.
- **arma_trend:** Gradient of the fitted trend, indicating trend momentum.
- **arma_detrended:** Original price with the long-term trend removed.

This module serves as an effective trend decomposition and denoising technique, particularly beneficial for tree-based ensembles.

The four modules provide complementary views of the weekly stock series: spectral gating describes frequency-domain energy and denoised trajectories, STL separates trend, seasonal behavior, and irregular deviations, LSTM-DAE learns nonlinear denoised price representations, and ARIMA captures linear trend and residual structure. They are evaluated with different classifiers to measure both their individual effects and their combined contribution to prediction performance. This modular design also makes it straightforward to substitute, compare, or ensemble denoising and trend-extraction techniques in future work.

5.2.5 Training Hyperparameters Details

Table 1: Training hyperparameters for experiments.

Group	Parameter	Value
General	Validation scheme	Walk-forward expanding window, 5 folds
	Sequence length	12 weeks
	Random seed	42
BiLSTM + Attention	BiLSTM units	48
	Dropout	0.3; recurrent dropout 0.1
	L2 regularization	1×10^{-3}
	Optimizer	Adam, learning rate 5×10^{-4}
	Batch size	16
	Maximum epochs	80 for Fold 0; 20 for fine-tuning
	Early stopping	Patience 10 for Fold 0; 5 for fine-tuning
Tree ensemble	LightGBM	500 estimators, learning rate 0.02
	Random Forest	300 estimators
ARIMA	Order	(1, 1, 1)
	Rolling window	104 weeks (2 years)
STL	Seasonal period	8 weeks and 13 weeks
	Seasonal window	7
	Iterations	5 inner, 20 outer
	Smoothing degree	1 (locally linear)
Spectral Gating	ECG setting	Beat-level and record-level STFT gating
	Stock setting	Causal rolling-window spectral features
	STFT mask	Soft mask with noise-floor thresholding
	Leakage control	Only historical data up to week t for stock features

5.3 Metrics Benchmarks

Model performance is evaluated using the standard classification metrics Accuracy, Precision, Recall, and F1-score. For a given class, let TP denote true positives, TN true negatives, FP false positives, and FN false negatives. Accuracy measures the overall proportion of correctly classified samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (6)$$

Precision measures how reliable the positive predictions are. A high precision means that when the model predicts a class, the prediction is likely to be correct:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (7)$$

Recall measures how many true samples of a class are successfully detected. A high recall

means that the model misses few positive cases:

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (8)$$

The F1-score combines precision and recall through their harmonic mean, making it useful when the dataset is imbalanced or when both false positives and false negatives are important:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (9)$$

For multi-class experiments, these metrics are computed for each class and then averaged. In particular, macro Precision, macro Recall, and macro F1 assign equal weight to every class, making them more informative than accuracy when class imbalance is present, such as in arrhythmia classification or directional stock movement prediction.

Before reporting the final results, we clarify that the model choice follows the structure of each dataset. For FPT stock data, tree ensembles are used because the inputs contain many engineered tabular features with nonlinear interactions, while BiLSTM + Attention is included to capture temporal dependencies across weekly observations. For MIT-BIH ECG, the main discriminative information lies in heartbeat morphology and RR dynamics, so sequence-oriented models and time-series denoising methods such as ARIMA and LSTM-based reconstruction are more suitable than purely tabular models.

6 Experiment Result

This section reports the comparative performance of the evaluated methods on the two datasets. Table 2 separates the results by dataset while keeping the same evaluation columns for direct comparison.

In Table 2, **Acc.** denotes Accuracy, **Prec.** denotes macro Precision, **Rec.** denotes macro Recall, and **F1** denotes macro F1-score.

Table 2: Experimental results by model and applied denoising method.

Dataset	Model	Applied method	Acc.	Prec.	Rec.	F1
MIT-BIH Arrhythmia	ARIMA	None (baseline)	0.45	0.80	0.45	0.54
	BiLSTM + Attention	None (baseline)	0.76	0.79	0.77	0.76
	BiLSTM + Attention	Median + bandpass filtering	0.80	0.83	0.81	0.80
	BiLSTM + Attention	Kalman filtering	/	/	/	/
	BiLSTM + Attention	Spectral gating (beat-level)	0.87	0.67	0.63	0.64
	BiLSTM + Attention	Spectral gating (record-level)	0.90	0.76	0.70	0.71
	BiLSTM + Attention	STL denoising (period = 1s)	0.82	0.82	0.82	0.81
	BiLSTM + Attention	ARIMA denoising	0.82	0.84	0.82	0.81
	BiLSTM + Attention	LSTM denoising	0.85	0.84	0.85	0.84
FPT Stock	Tree Ensemble	None (baseline)	0.40	0.39	0.38	0.38
	BiLSTM + Attention	None (baseline)	0.62	0.63	0.63	0.62
	Tree Ensemble	Median Filtering	0.45	0.44	0.43	0.43
	BiLSTM + Attention	Median Filtering	0.46	0.51	0.52	0.45
	Tree Ensemble	Adaptive Kalman filtering	0.56	0.56	0.56	0.55
	BiLSTM + Attention	Adaptive Kalman filtering	0.50	0.56	0.55	0.49
	Tree Ensemble	Spectral features	0.38	0.36	0.36	0.36
	BiLSTM + Attention	Spectral features	0.44	0.46	0.46	0.44
	Tree Ensemble	STL features (period = 8)	0.54	0.52	0.53	0.52
	Tree Ensemble	STL features (period = 13)	0.56	0.53	0.52	0.52
	BiLSTM + Attention	STL features (period = 8)	0.72	0.71	0.72	0.71
	BiLSTM + Attention	STL features (period = 13)	0.58	0.58	0.58	0.57
	Tree Ensemble	ARIMA features	0.34	0.36	0.36	0.33
	BiLSTM + Attention	ARIMA features	0.64	0.68	0.67	0.64
	Tree Ensemble	LSTM denoising	0.74	0.73	0.72	0.72
	BiLSTM + Attention	LSTM denoiser	0.76	0.76	0.73	0.74

The results in Table 2 show that denoising performance depends strongly on the dataset, the classifier, and the evaluation metric. On MIT-BIH, the standalone ARIMA baseline performs poorly, with accuracy 0.45 and F1-score 0.54, mainly because its recall is only 0.45, showing that a purely linear statistical model is not sufficient for heartbeat classification. The BiLSTM + Attention baseline is much stronger, reaching accuracy 0.76 and F1-score 0.76, because it can exploit waveform morphology and temporal dependencies. Median + bandpass filtering further improves the BiLSTM baseline to accuracy 0.80 and F1-score 0.80, confirming that standard ECG filtering is a useful preprocessing step. Kalman filtering has no reported metrics in the table, reflecting the fact that this configuration was not retained for final quantitative comparison. Among the remaining denoising methods, ARIMA denoising and STL denoising both provide moderate gains, each reaching accuracy 0.82 and F1-score 0.81. LSTM denoising gives the most balanced ECG result, with accuracy 0.85, recall 0.85, and the best F1-score 0.84, suggesting that learned reconstruction preserves discriminative heartbeat morphology better than linear smoothing or decomposition alone. Spectral gating gives a mixed result: beat-level spectral gating reaches accuracy 0.87 but only F1-score 0.64, while record-level spectral gating achieves the highest accuracy 0.90 but still only F1-score 0.71. This gap shows that high accuracy can be misleading under class imbalance; spectral gating likely improves recognition

of dominant heartbeat types but remains weaker on minority arrhythmia classes, as reflected by its lower macro precision and recall.

For the FPT stock dataset, the two baselines are Tree Ensemble with accuracy 0.40 and F1-score 0.38, and BiLSTM + Attention with accuracy 0.62 and F1-score 0.62. Adaptive Kalman filtering improves the Tree Ensemble to accuracy 0.56 and F1-score 0.55, but it reduces BiLSTM + Attention performance to accuracy 0.50 and F1-score 0.49. This contrast suggests that Kalman smoothing can help tabular models by producing cleaner price-derived indicators, but may remove short-term temporal fluctuations that a sequence model can use. Median filtering gives a weaker but still interpretable result. It reaches 0.45 for the accuracy and 0.43 for F1-score with Tree Ensemble, and 0.46, 0.45, respectively, with BiLSTM + Attention. Median filtering only gives limited improvement because it smooths isolated spikes, but in FPT stock data, sudden weekly changes may be real market signals rather than noise. Therefore, it can make technical indicators more stable, but may also remove useful short-term information for next-week direction prediction. ARIMA features hurt the Tree Ensemble, dropping it to accuracy 0.34 and F1-score 0.33, but slightly improve BiLSTM + Attention to accuracy 0.64 and F1-score 0.64, indicating that linear trend-residual information is more useful when combined with a sequential model than when used in the tree-based feature set.

STL features show strong sensitivity to period selection and model type. For the Tree Ensemble, STL period 8 reaches accuracy 0.54 and F1-score 0.52, while period 13 gives slightly higher accuracy 0.56 but the same F1-score 0.52. For BiLSTM + Attention, the difference is much larger: period 13 performs below the baseline with accuracy 0.58 and F1-score 0.57, whereas period 8 improves substantially to accuracy 0.72 and F1-score 0.71. LSTM denoising is the strongest FPT method for both classifiers, reaching accuracy 0.74 and F1-score 0.72 with Tree Ensemble, and accuracy 0.76 and F1-score 0.74 with BiLSTM + Attention. In contrast, spectral features are weak for both FPT models, with F1-score 0.36 for Tree Ensemble and 0.44 for BiLSTM + Attention, confirming that frequency-domain descriptors are less suitable for irregular, non-stationary stock movements. Overall, the table indicates that spectral methods are more suitable for structured ECG signals than for financial data, median filtering behaves as a weak robust-smoothing baseline, Kalman filtering is more helpful for tabular models than sequence models, STL depends strongly on period choice, and LSTM denoising is the most consistently effective method across both domains.

7 Conclusion

This report investigated time series noise reduction on two contrasting datasets: MIT-BIH ECG signals and FPT weekly stock data. The experiments show that no denoising method is universally optimal. Instead, each method is useful under different assumptions: ARIMA is interpretable but limited for nonlinear patterns, STL can extract useful trend and seasonal structure when the period is appropriate, Kalman filtering can reduce noise for tabular feature-based models but may over-smooth sequences, spectral gating is effective when signal structure

is stable in the frequency domain, and LSTM denoising is more flexible for nonlinear temporal reconstruction.

The most important finding is that evaluation must consider more than accuracy. On MIT-BIH, record-level spectral gating achieved the highest accuracy (0.90), but its F1-score (0.71) was much lower than LSTM denoising (0.84), indicating weaker performance on minority arrhythmia classes. For FPT, LSTM denoising was the strongest method for both Tree Ensemble and BiLSTM + Attention, reaching F1-scores of 0.72 and 0.74, respectively. These results suggest that useful denoising should preserve task-relevant temporal information rather than simply make the signal smoother.

The study still has several limitations. Some methods were evaluated with fixed configurations, such as ARIMA orders, STL periods, Kalman state assumptions, and spectral settings, so more systematic tuning may change the results. The MIT-BIH evaluation is affected by class imbalance, while the FPT stock task is inherently unstable because market movements depend on regimes, news, liquidity, and other external factors that are not fully represented in the current features. Future work should therefore focus on three directions: extending the experimental setup to more datasets, applying more complex imbalance-handling methods, and studying denoising under multi-objective problems where accuracy, robustness, interpretability, and computational cost must be considered together. In summary, denoising should be selected as part of the modelling pipeline, with attention to the signal domain, classifier behavior, and target metric.

References

- [1] H. Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974.
- [2] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung. *Time Series Analysis: Forecasting and Control*. Wiley, 5th edition, 2015.
- [3] P. J. Brockwell and R. A. Davis. *Introduction to Time Series and Forecasting*. Springer, 3rd edition, 2016.
- [4] J. D. Hamilton. *Time Series Analysis*. Princeton University Press, 1994.
- [5] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin. Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54(1–3):159–178, 1992.
- [6] G. M. Ljung and G. E. P. Box. On a measure of lack of fit in time series models. *Biometrika*, 65(2):297–303, 1978.
- [7] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

- [8] S. J. Julier and J. K. Uhlmann. A new extension of the Kalman filter to nonlinear systems. In *Proceedings of SPIE 3068, Signal Processing, Sensor Fusion, and Target Recognition VI*, pages 182–193, 1997.
- [9] D. Simon. *Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches*. Wiley-Interscience, Hoboken, NJ, 2006.
- [10] H. Lutkepohl. *New Introduction to Multiple Time Series Analysis*. Springer, 2005.
- [11] G. Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.
- [12] R. H. Shumway and D. S. Stoffer. *Time Series Analysis and Its Applications*. Springer, 2000.
- [13] R. S. Tsay. *Analysis of Financial Time Series*. Wiley, 3rd edition, 2010.
- [14] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning. STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6(1):3–73, 1990.